



# Docker Study Guide

A personal, hands-on study guide built while learning Docker from a YouTube course. Each topic is taught in **3 stages** — Beginner → Working Developer → Expert — plus an **Interview Corner**. Examples are tailored to a **JavaScript / React / Next.js / MongoDB / Prisma** stack.

**Learner:** Akshay **Started:** 6 August 2026

## How to use this guide

Every topic follows the same structure:

1. **Beginner** — plain-English "what is this and why do I care," with an analogy.
2. **Working Developer** — how it works, the real commands/syntax, how you'd use it in a project.
3. **Expert** — deeper mechanics, edge cases, best practices, and the "why."
4. **Interview Corner** — likely questions, crisp answers, and traps to avoid.

## Table of Contents

1. [What is Docker?](#)
2. [What is a Container and an Image?](#)

## 1. What is Docker?

### Beginner

Docker is a tool that packages your application together with everything it needs to run — the code, the runtime (like Node.js), libraries, and settings — into a single sealed box called a **container**. That container then runs the exact same way on any computer.

It solves the classic *"but it works on my machine!"* problem. Your app might work on your laptop but break on a teammate's machine due to a different Node version, a missing env variable, or a different OS. The code is the same; the *environment* differs.

**Analogy — the shipping container** 🚢: Before standardized shipping containers, loading a ship was chaos (odd-shaped boxes, barrels, sacks). The container standardized one box shape that any ship, crane, or truck can handle

regardless of contents. Docker does this for software: once your app is in a container, any machine with Docker runs it identically.

**One line:** Build your app once, run it anywhere, exactly the same way.

## 🟡 Working Developer

Docker is a **containerization platform**. Core pieces:

- **Image** — a read-only blueprint/template: app + dependencies frozen into a package (e.g. `node:20`, or your own `my-nextjs-app`). Like a *class*.
- **Container** — a running instance of an image. Like an *object*. One image → many containers.
- **Dockerfile** — a text file of instructions telling Docker how to build an image.
- **Docker Engine** — the background daemon that builds images and runs containers.
- **Docker Hub** — a public registry (GitHub for images) to pull ( `node`, `mongo`, `nginx` ) or push images.

Typical flow (JS stack):

```
# Build an image from a Dockerfile
docker build -t my-next-app .

# Run a container from that image
docker run -p 3000:3000 my-next-app

# Need MongoDB? No local install needed
docker run -d -p 27017:27017 mongo
```

The win: you get a full MongoDB running **without installing it on your machine**. Delete the container when done and your machine stays clean.

## 🔴 Expert

**Docker is NOT a virtual machine.** A VM virtualizes *hardware* and runs a full guest OS (its own kernel) via a hypervisor — heavy (GBs of RAM, boots in minutes). A container **shares the host's Linux kernel** and isolates only the *process* — lightweight (MBs), starts in milliseconds.

Isolation comes from Linux kernel features:

- **Namespaces** — each container gets its own isolated view (processes, network, filesystem, hostname).
- **cgroups** — limit/meter resources (e.g. max 512MB RAM, 1 CPU per container).
- **Union filesystem (layers)** — images are stacked, cached, read-only layers; shared base layers (e.g. `node:20`) are stored once. Fast builds, efficient storage.

**Nuance:** Linux containers need a Linux kernel, so on Windows/Mac, Docker Desktop runs a lightweight Linux VM underneath to provide one.

**Why it matters:** Containers make apps reproducible (same artifact from laptop → CI → prod), scalable (spin up N identical containers), and portable (move across clouds). They're the foundation of microservices and orchestration tools like Kubernetes.

## Interview Corner

- **What is Docker (one line)?** A platform that packages an app + its dependencies into a lightweight, portable container that runs consistently across any environment.
- **What problem does it solve?** The *"works on my machine"* problem — environment inconsistency between dev, test, and prod.
- **Container vs VM?** VM virtualizes hardware + full guest OS/kernel (heavy, slow). Container shares host kernel, isolates the process (light, ms startup). VM = stronger isolation; container = speed/efficiency.
- **Image vs Container?** Image = read-only blueprint; container = running instance of it. One image → many containers.
- **⚠ Traps:** Don't call Docker a VM (#1 wrong answer). Don't confuse image vs container. Remember: containers share the host kernel.

## 2. What is a Container and an Image?

### Beginner

An **image** is a *blueprint*; a **container** is a *running thing built from that blueprint*.

- **Image** — a read-only package containing your app + everything it needs (code, runtime, libraries, config). It sits on disk and does nothing by itself. Like a **recipe** or a house blueprint.
- **Container** — what you get when you *run* an image. It's alive, executing your app. Like the **cooked dish** made from the recipe.

From one recipe you can cook the dish many times → **from one image you can run many containers**, all identical and independent.

Developer phrasing: **Image = class** (definition), **Container = object/instance**.

### Working Developer

```
docker pull node:20          # download the image (blueprint)
docker images                # list images on disk
docker run -d --name web1 -p 3000:3000 node:20 # create+start a container
docker run -d --name web2 -p 3001:3000 node:20 # another, independent container
docker ps                    # list running containers
docker stop web1 && docker rm web1             # stop + remove a container
```

One image ( `node:20` ) → two separate containers ( `web1` , `web2` ), each with its own isolated filesystem, processes, network.

Images come from a **Dockerfile**:

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]    # what runs when a CONTAINER starts
```

```
docker build -t my-next-app .    # Dockerfile -> Image
docker run -p 3000:3000 my-next-app    # Image -> Container
```

**Lifecycle:** `Dockerfile` → (build) → **Image** → (run) → **Container**

**Containers are ephemeral:** data written inside a container is lost when it's deleted (it wasn't part of the read-only image). Use **volumes** to persist data (e.g. MongoDB data).

## 🔴 Expert

- **Images = stacked read-only layers.** Each Dockerfile instruction ( `FROM` , `COPY` , `RUN` ) adds a cached, shareable layer. A shared base ( `node:20` ) is stored on disk once. Rebuilds reuse cached layers, so instruction order matters (copy `package.json` + install deps before copying all code).
- **A container = image layers + a thin writable layer** added on top via **copy-on-write**. Reads come from shared read-only layers; writes go only to the container's private top layer. Deleting a container discards only that thin layer; the image is untouched.
- **Formula:** *Container = Image + writable layer + running process.*
- **Stopped ≠ removed:** a stopped container keeps its writable layer until `docker rm` . `docker run` always makes a new container; use `docker start <name>` to restart an existing one.
- **Image identity = content hash** ( `sha256:...` ); tags like `node:20` or `latest` are movable pointers. Pin versions/digests in production for reproducibility ( `latest` is not a fixed version).

## 🎯 Interview Corner

- **Image vs Container?** Image = read-only blueprint of app + deps; Container = running instance of an image with an added writable layer. One image → many containers.
- **Analogy?** Image:Container = Class:Object = Recipe:Dish.
- **Can one image create many containers?** Yes — any number, each isolated, sharing the same read-only layers.
- **Data written in a container when removed?** Lost — it lived in the writable layer. Persist via **volumes**/bind mounts.

- **What are image layers & why do they matter?** Stacked read-only layers (one per instruction), cached and shared → saves storage, speeds builds. Good ordering maximizes cache hits.
- **docker run vs docker start ?** `run` creates a new container from an image; `start` restarts an existing stopped container.
- **⚠ Traps:** "An image is a running container" (reversed); assuming data persists by default (it doesn't); treating `latest` as a fixed version.

## 🔧 Command Deep-Dive: `docker run` flags (from Q&A)

Line-by-line breakdown of the core workflow commands:

```
docker pull node:20           # download image (repo=node, tag=20=version)
docker images                 # list images on disk
docker run -d --name web1 -p 3000:3000 node:20 # create+start a container
docker run -d --name web2 -p 3001:3000 node:20 # a second, independent container
docker ps                     # list running containers (-a = all)
docker stop web1              # stop (container still exists)
docker rm web1                # delete container permanently
```

| Piece                         | Meaning                                                                                                                                                                                                         |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>node:20</code>          | image name: <code>node</code> = repository, <code>20</code> = tag/version. No tag → <code>latest</code> .                                                                                                       |
| <code>pull</code>             | download an image from Docker Hub to disk                                                                                                                                                                       |
| <code>docker images</code>    | list images stored locally                                                                                                                                                                                      |
| <code>docker run</code>       | create a <b>new</b> container and start it                                                                                                                                                                      |
| <code>-d</code>               | <b>detached</b> — run in background, return the terminal                                                                                                                                                        |
| <code>--name web1</code>      | give the container a <b>custom, memorable name</b> (else Docker picks a random one like <code>sleepy_einstein</code> ). Must be unique; used later in <code>stop</code> / <code>logs</code> / <code>rm</code> . |
| <code>-p 3000:3000</code>     | <b>port map</b> <code>HOST:CONTAINER</code> — traffic on host port 3000 → container port 3000. Open <code>localhost:3000</code> to reach the app.                                                               |
| <code>docker ps</code>        | list running containers; <code>docker ps -a</code> lists all (incl. stopped)                                                                                                                                    |
| <code>docker stop web1</code> | stop a running container (it still exists on disk)                                                                                                                                                              |
| <code>docker rm web1</code>   | permanently delete a container ( <code>-f</code> to force-remove a running one)                                                                                                                                 |

**Why `web2` uses `-p 3001:3000`** : two containers can't share the same **host** port, so the host side differs (3000 vs 3001). The **container** side stays 3000 because each container is isolated — no clash. Result: `localhost:3000` →

web1, `localhost:3001` → web2.

**Names must be unique:** you can't have two live containers both named `web1`. `--name` applies to the *container*, not the image.

---